Guardiant

SaveMe Protocol — End-to-End Technical Preparation Sheet

Every component, technology choice, design decision, failure and fix in the project, written to be studied before a demo, viva or interview. Sourced directly from the repository (**Guardiant-main**, branch `deploy-ready`), its commit history, DEPLOYMENT.md, the Ignition deployment journal and the live Vercel project.

Item	Value
Project	Guardiant, a.k.a. the SaveMe Protocol: a Web3 rug-pull protection dApp
Repo layout	`client/` Next.js 14 dApp · `contract/` Hardhat + Solidity 0.8.28 · `model/` FastAPI + scikit-learn ML service
Timeline	29 Apr 2025: initial protocol (contracts, client, model) · 5 May 2025: architecture docs · 16 Aug 2026: deployability, detection accuracy, 26-issue security audit, HF deploy script
Live frontend	Vercel project `guardiant` · production: guardiant-knr80sv9t-deepeshs-projects-b1b35077.vercel.app
Contracts	Deployed on Ethereum Sepolia (chain 11155111); final addLiquidity confirmed in block 11,501,484
ML service	Docker image ready for Hugging Face Spaces; not live yet (blocked on `hf auth login`)
Size	~6,700 lines of TS/TSX in client/src, 6 Solidity contracts, ~15 Python modules, 13 regression tests

Contents

1. The pitch (30 s / 2 min)
2. Problem and solution
3. System architecture
4. Tech stack and why each piece
5. Smart contracts in depth
6. Contract deployment (Ignition, Sepolia, gas)
7. Frontend: pages, hooks, state, wallet
8. Frontend design system
9. Next.js API routes (the server layer)
10. ML anomaly-detection service
11. End-to-end user flows
12. Failures and how each was solved
13. Security audit: all 26 findings
14. Testing and verification
15. Deployment and hosting
16. Known limitations (be honest about these)
17. Likely interview questions with answers
18. Glossary and numbers to remember

1. The pitch

30 seconds

Guardiant is a Web3 protocol that protects users from rug pulls. A Python ML service scores every wallet transaction with an Isolation Forest plus percentile threshold rules, the Next.js dApp shows the risk in real time, and a guarded smart-contract wallet has an emergency exit that converts the user's tokens to ETH through our own AMM and returns them to the owner, and that exit is written so it can never revert.

2 minutes

Rug pulls happen fast: a token's liquidity is drained or a huge transfer leaves a wallet, and by the time a human reacts the funds are gone. Guardiant has three parts. **(1) Contracts on Sepolia**: a custodial Wallet ledger, a TokenFactory that mints ERC-20s, a constant-product ETH/token LiquidityPool, a TransferTo helper and the AnomalyGuardWallet with an unblockable emergency exit. **(2) A Next.js 14 dApp on Vercel** with RainbowKit/wagmi wallet connection, token creation with metadata in MongoDB, a natural-language transfer box parsed by Gemini, a transaction dashboard with anomaly badges, and a rug-pull market simulation. **(3) A FastAPI service** in Docker that cleans transactions, builds five features, log-scales and standardises them with a scaler fitted at training time, and unions an Isolation Forest verdict with 99.5th-percentile rules. It catches 3/3 obvious attacks at a 3% false-positive rate on held-out normal traffic.

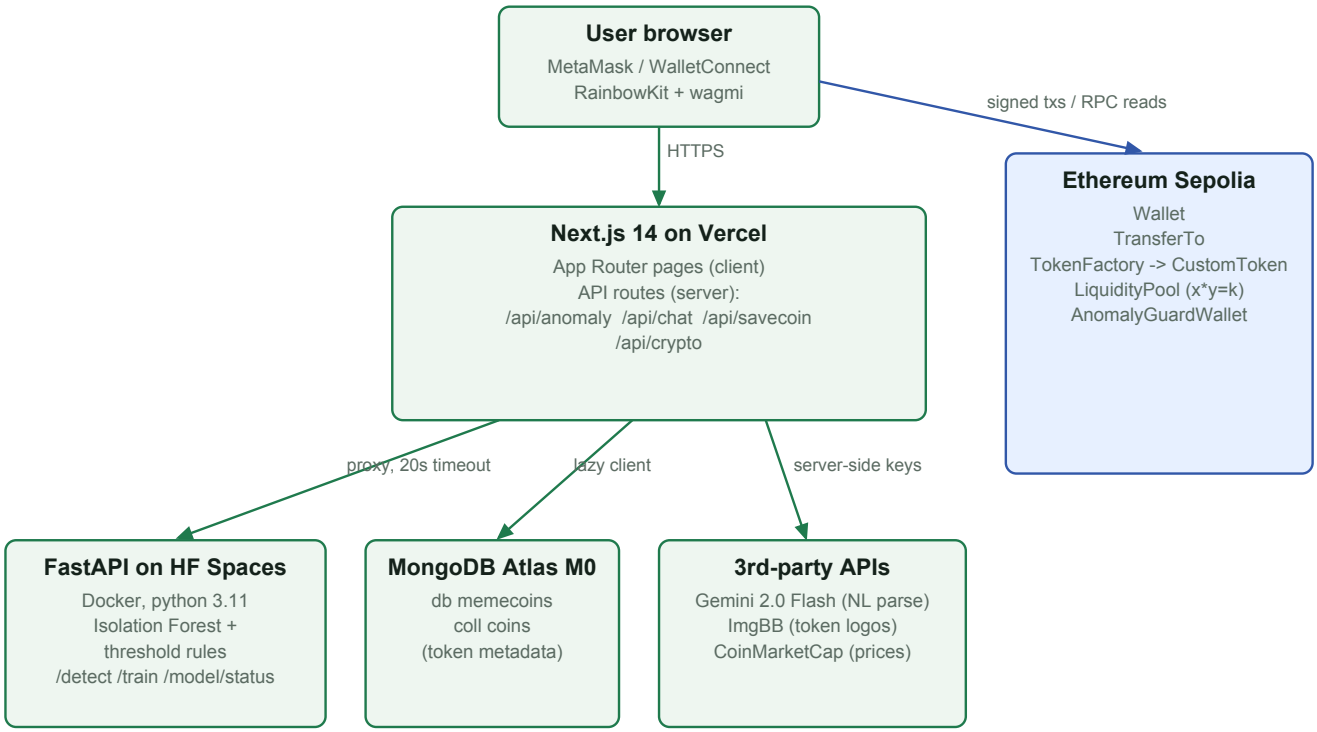
The interesting engineering is in what broke: the app couldn't build, the ML returned 200s but detected nothing, and an audit found two unconditional fund drains in the pool. Section 12 and 13 walk through every one.

2. Problem and solution

Problem	How Guardiant answers it
Rug pulls: liquidity pulled, huge drains, approval abuse	Per-transaction anomaly scoring; zero-value approvals are explicitly scored, not dropped
Users react too slowly	Real-time anomaly badges in /transactions; one-call emergency exit in AnomalyGuardWallet
Exits fail exactly when needed (pool too shallow, contract wallets)	Swap wrapped in try/catch, falls back to sending raw tokens; ETH paid with call{value:} not transfer()
Crypto UX is intimidating	Type 'send 0.01 ETH to vitalik.eth'; Gemini extracts the fields, viem validates the address
Hard to understand what a rug pull looks like	/graph simulates a market with whale, bot, retail and manipulator agents and a scheduled rug pull

Core features (README): real-time anomaly detection, automatic self-destruct / exit mechanism, instant liquidity conversion and fund recovery, protected wallet creation, transaction monitoring dashboard. **User actions:** connect wallet, create protected wallet, transfer funds, monitor wallet, view transactions.

3. System architecture



Browser never sees service URLs or API keys; all secrets live in Vercel server env.

Design principles behind the shape

- **Three independently deployable pieces, three free hosts.** Vercel Hobby (frontend), Hugging Face Spaces Docker (ML), Sepolia (contracts). Total cost: $0/month.
- **Server-side proxy for every secret or third-party call.** `/api/anomaly` forwards to FastAPI, so the browser never learns the service URL, and CORS and mixed-content never arise. Gemini, ImgBB, CoinMarketCap and Mongo keys never use a `NEXT_PUBLIC_` prefix.
- **Wallet writes go straight from browser to chain.** The server never holds a user key; the user's wallet signs every transaction.
- **Graceful degradation.** ML down → the transaction list still renders with an empty anomaly map. CoinMarketCap down → prices flagged `fallback: true`. ImgBB down → token saved without image.
- **Configuration by environment, not code.** Chain id, RPCs, contract addresses and the WalletConnect id all come from env vars; `deploy.sh` writes them for you.

4. Tech stack and why each piece

Layer	Technology	Why it was chosen
Contracts	Solidity 0.8.28	Built-in checked arithmetic (no SafeMath), custom errors and try/catch on external calls, which the emergency exit relies on
	Hardhat 2.22 + hardhat-toolbox	JS-native toolchain with ethers, chai, a local node, Etherscan verify and gas reporting in one package
	Hardhat Ignition	Declarative deployment: dependency graph, resumable journal (journal.jsonl), reads event args from earlier steps
	OpenZeppelin 5.2	Audited ERC20, SafeERC20, ReentrancyGuard and Math.sqrt instead of hand-rolling security primitives
	Sepolia testnet	Long-lived Ethereum testnet with free faucets and Etherscan support
Frontend	Next.js 14 (App Router)	One codebase for React pages and serverless API routes, so secrets stay server-side; first-class on Vercel
	React 18 + TypeScript 5.5	Type safety across ABIs, hooks and API payloads

Layer	Technology	Why it was chosen
	wagmi 2 + viem 2	Typed React hooks for reads/writes/events; viem infers types from `as const` ABIs and is far lighter than ethers
	RainbowKit 2	Polished connect modal for MetaMask, WalletConnect, Coinbase and more, plus chain switching, out of the box
	TanStack Query 5	Required by wagmi; caching and dedup for chain reads
	ethers 6	Used on /liquidity for `BrowserProvider` signer + `staticCall` quoting
	Vercel AI SDK + @ai-sdk/google	`generateObject` with a zod schema returns typed JSON from Gemini; no hand-parsing of LLM text
	zod 3	Single validation library for every API body and for the LLM output schema
	MongoDB driver 6 / Atlas M0	Schemaless token metadata (name, symbol, supply, image URL); free 512 MB cluster
	Tailwind CSS 4 + shadcn/ui	Utility CSS with `@theme` tokens; Radix-based Button/Tabs with CVA + tailwind-merge
	motion 12, react-spring 9	Landing page text animations (BlurText, CircularText, RotatingText)
	react-hot-toast, react-icons, lucide	Notifications and icons
ML	Python 3.11	pandas 2.1.1 / numpy 1.24.3 pins have wheels for 3.11 but not 3.13+
	FastAPI 0.104 + Pydantic 2 + uvicorn	Typed request validation with length bounds, async, auto OpenAPI docs at /docs
	scikit-learn 1.3.1 IsolationForest	Unsupervised: no labelled rug-pull dataset exists; fast, handles mixed scales once log-scaled
	pandas, numpy, joblib	Feature frame, maths, model/scaler/threshold persistence
	requests + Etherscan API	Optional training on a real wallet's history (`POST /train` with no body)
Infra	Docker (python:3.11-slim)	Reproducible image; the model is trained during the build so artifacts match the installed sklearn
	Vercel / HF Spaces / Atlas / Reown	All free tiers, none needs a card

5. Smart contracts in depth

Six contracts in `contract/contracts/`. All ETH payouts use `call{value:}` with a success check, all token moves use SafeERC20, and every state-changing entry point that makes external calls is `nonReentrant`.

5.1 LiquidityPool.sol: constant-product ETH/token AMM

- **Pricing:** Uniswap-v2-style x·y=k with a 0.3% fee applied to the input:

```
amountOut = (amountIn * 997 * reserveOut) / (reserveIn * 1000 + amountIn * 997)
```

- Output is **strictly less than reserveOut** for any finite input, so no swap can empty a reserve.
- **Reserves tracked in storage** (`tokenReserve`, `ethReserve`), not read from `balanceOf` or `address(this).balance`. Donating assets directly cannot move the price; `receive()` accepts ETH without counting it.
- **LP shares, not token amounts.** First deposit mints `sqrt(tokens × eth)` shares minus `MINIMUM_LIQUIDITY = 1000`, locked forever so `totalLiquidity` can never return to 0 (prevents re-seeding at an arbitrary price). Later deposits mint `min(tokens·T/tokenReserve, eth·T/ethReserve)`, so the limiting side sets the count.
- **Fee-on-transfer safe:** measures `balanceAfter − balanceBefore` and credits only what actually arrived.
- **Slippage bounds on everything:** `addLiquidity(tokenAmount, minShares)`, `removeLiquidity(shares, minToken, minEth)`, `swap(tokenAmount, minEthOut)`, `swapEthForTokens(minTokensOut)`.
- **Views:** `getSwapRate` / `getTokenSwapRate` return 0 on an empty pool instead of reverting.

- **Events:** LiquidityAdded, LiquidityRemoved, Swapped.

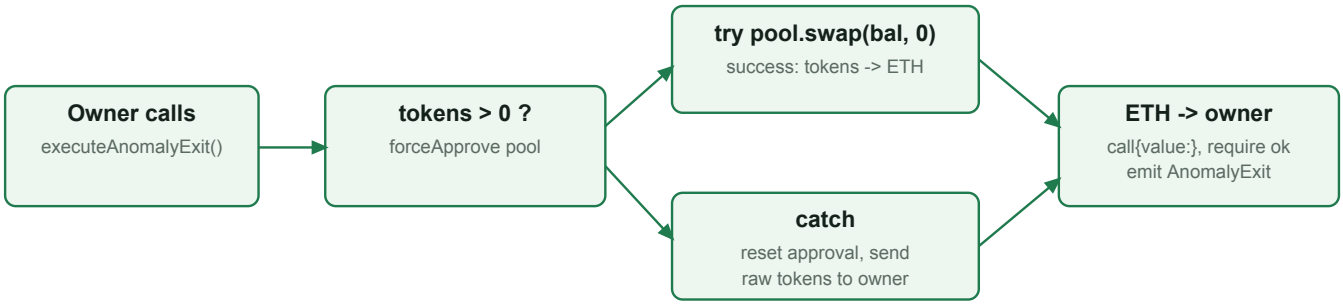
Worked example: why the old linear pricing was a drain

The deployed pool is seeded with 10,000 GDT and 0.02 ETH. The old formula `ethOut = tokenIn × ethReserve / tokenReserve` gives, for tokenIn = 10,000: 10,000 × 0.02 / 10,000 = **0.02 ETH, i.e. 100% of the pool**. With constant product: 10,000·997·0.02 / (10,000·1000 + 10,000·997) ≈ **0.00999 ETH (≈ 50%)**, and to take 99% you'd need roughly 100× the reserve. Price impact grows without bound, which is the whole point of x·y=k.

Worked example: the 1-wei LP exploit

Old code tracked positions in token units only. Alice deposits 10,000 tokens + 0.02 ETH. An attacker deposits 10,000 tokens + 1 wei and owns 50% of the "liquidity", then withdraws half of everything, about 0.01 ETH of Alice's. New code mints `min(10,000·T/10,000, 1wei·T/0.02ETH)` ≈ 0 shares and reverts with "Zero shares minted".

5.2 AnomalyGuardWallet (AnomalyGuard.sol): the emergency exit



- Holds an immutable `token` and `liquidityPool`. Owner-only: `forwardTokens`, `forwardETH`, `swapETHForTokens`, `executeAnomalyExit`, `rescueToken`, `transferOwnership`.
- **executeAnomalyExit:** `forceApprove` the pool, then `try liquidityPool.swap(balance, 0)`. On failure (empty pool, too large, paused) it resets the approval and sends the raw tokens to the owner. Then all ETH goes out via `call`. Emits `AnomalyExit(tokensSwapped, ethReturned, swapSucceeded)`.
- **Deliberate trade-off: minEthOut = 0.** This is the panic path. Getting out matters more than price; front-running costs value but cannot block the exit, and a reverting exit is the worse failure.
- `forceApprove` (OZ 5) handles tokens like USDT that require resetting allowance to 0 first.
- **Name note:** the file is AnomalyGuard.sol, the contract is `AnomalyGuardWallet`.

5.3 Wallet.sol: custodial ledger

- `addToken(token, amount)` pulls tokens in with safeTransferFrom and credits only what arrived. `removeToken` pays out. `transferToken` moves balances inside the ledger (no external call, no reentrancy surface).
- O(1) membership with `tokenTracked[user][token]`, so `userTokens` never gets duplicates. Events: Deposited, Withdrawn, InternalTransfer.

5.4 TokenFactory.sol + CustomToken.sol

- `createToken(name, symbol, initialSupply)` deploys a new `CustomToken` (OZ ERC20) that mints `initialSupply × 10^decimals` to the caller, and emits `TokenCreated(tokenAddress, name, symbol, initialSupply, creator)`. The frontend reads that event to learn the new address.
- Because the token scales by 10^18 internally, callers pass a **plain count** (`1_000_000n`), not `parseEther`. Passing parseEther was the 10^42-supply bug.

5.5 TransferTo.sol

- `sendToken(token, recipient, amount)` uses safeTransferFrom from caller to recipient. The old pre-flight `balanceOf` check was removed: it duplicated what transferFrom enforces while ignoring allowance,

the case that actually fails.

6. Contract deployment

One Ignition module, `ignition/modules/Guardiant.js` (`GuardiantModule`), replaced six per-contract modules that hardcoded Hardhat local addresses. Order:

```
Wallet -> TransferTo -> TokenFactory
  -> tokenFactory.createToken('Guardiant Demo Token','GDT', 1_000_000n)
  -> readEventArgument(createToken, 'TokenCreated', 0)  => token address
  -> LiquidityPool(token)
  -> token.approve(pool, 10,000 GDT)
  -> pool.addLiquidity(10,000 GDT, minShares 0) {value: 0.02 ETH}   (after: approve)
  -> AnomalyGuardWallet(token, pool)
```

`deploy.sh [localhost|sepolia]` runs the deploy, reads `deployed_addresses.json` and writes all six addresses plus `NEXT_PUBLIC_DEFAULT_CHAIN_ID` into `client/.env.local`, keeping existing secrets. `hardhat.config.js` uses the publicnode Sepolia RPC by default, `DEPLOYER_PRIVATE_KEY` from .env, and Etherscan for verification.

Contract	Sepolia address
Wallet	0xBf0AE445Ee0581048c047FC95894A26DAB76FeBb
TransferTo	0xdf8f44F3dD9C7D3977f5e29FFd916384cbB595F2
TokenFactory	0x4Bd3D446f80B8B28C40FA2875cF910b000472D5E
CustomToken (GDT)	0x5c6f3410Ddf6D371ae0F17Ed932136A9A79357bd
LiquidityPool	0x93EF97D253D3711DE7fEf5087A7E89019B4BA513
AnomalyGuardWallet	0x078c4F99DB588f369C4FDE4f70F1289b7b20f12D

Gas budgeting: `scripts/gas-estimate.js` replays the full deploy on a local chain and prints real gas per step (≈ **7.49M gas total**), then the ETH cost at 1/3/10 gwei plus the 0.02 ETH LP seed. Reason: guessing funding is how a testnet deploy dies halfway. The LP seed was cut from 10 ETH to 0.02 ETH so a faucet-funded key can finish.

7. Frontend: pages, hooks, state, wallet

7.1 Provider tree (`app/providers.tsx`)

```
WagmiProvider(config) > QueryClientProvider > RainbowKitProvider > WalletProvider(useAccount) > Navbar + page
```

`wagmi.ts` uses RainbowKit `getDefaultConfig` with a custom Hardhat chain (31337) plus Sepolia, Base Sepolia, mainnet, Polygon, Optimism, Arbitrum and Base. The chain from `NEXT_PUBLIC_DEFAULT_CHAIN_ID` is put first so RainbowKit defaults to it. Transports: Hardhat local, Sepolia and Base Sepolia with optional custom RPC. `cookieStorage` + `ssr: true` keep the config safe during server prerender (there is no localStorage on the server).

7.2 Pages

Route	What it does	Key mechanics
/	Landing: 'Save your funds from rug-pull', Connect button, how-it-works	BlurText, CircularText 'SELF*DESTRUCTING*CONTRACTS*', RotatingText
/wallet	Protected wallet: deposit, withdraw, balances	Wallet ABI; balances via a single viem multicall (balanceOf/symbol/decimals); refreshes on Deposited/Withdrawn events
/transfer	Natural-language send	POST /api/chat → Gemini fields → viem isAddress → useSendTransaction

Route	What it does	Key mechanics
/transactions	History + risk	useTransactionHistory (last 10 blocks, concurrent getBlock) → useAnomalyDetection → TransactionList badges, per-type breakdown, service status
/create	Create ERC-20	TokenFactory.createToken via wagmi writeContract → wait receipt → parse TokenCreated → POST /api/savecoin with metadata + base64 logo
/tokens	Token gallery	GET /api/savecoin (Mongo, limit 500)
/liquidity	Add liquidity	ethers BrowserProvider signer; approve → addLiquidity.staticCall quote → minShares = 99% of quote → send
/graph	Rug-pull simulator ('Chart')	useTradingAgents: whale/bot/retail/manipulator agents (momentum, contrarian, random, manipulative), candlesticks, scheduled rug pull with countdown. Client-side only

7.3 Hooks

- `useTransactionHistory`: reads the head with `getBlockNumber()` inside the fetch, fetches the last 10 blocks with `Promise.all`, filters txs to/from the user, and carries `valueWei, gas, gasPrice` for the model. Polls every 30 s.
- `useAnomalyDetection`: posts visible txs to /api/anomaly, returns a `hash → {is_anomaly, anomaly_types}` map, re-runs only when the joined hash key changes, cancels stale responses, degrades to an empty map on error.
- `useTokenCreation`, `useContractFunctions`, `useWallet`, `useCryptoPrice` (surfaces the `fallback` flag), `useTradingAgents`.
- All addresses centralised in `lib/contract.ts` with an `addr(env || fallback)` helper; Hardhat deterministic addresses kept only as dev defaults. ABIs in `lib/abi/*.ts` are `as const` so viem infers event and argument types.

8. Frontend design system

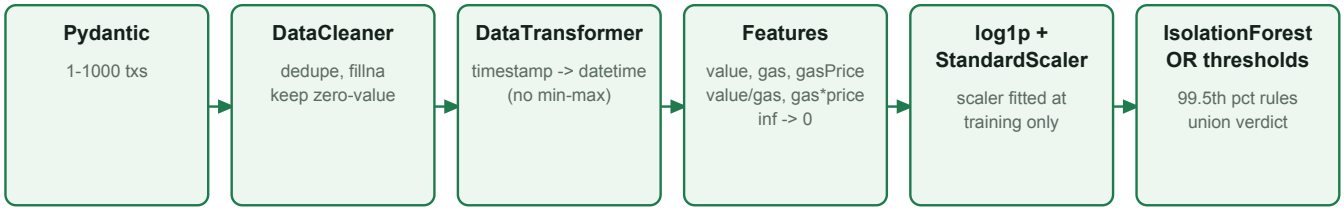
Element	Choice	Rationale
Theme	Dark only: background #051008 (near-black green), accent #8ddca4 (mint)	Security/finance feel; green = safe; high contrast for numbers and badges
Type	SF Display Pro → system stack; SF Mono for addresses/hashes	Native-feeling, zero font download; mono keeps hex aligned
Tokens	Tailwind v4 `@theme` (font, weights 500-900, colors) + shadcn oklch variables	One source of truth for colour and type
Components	shadcn/ui Button, Tabs (Radix), CVA variants, tailwind-merge, clsx	Accessible primitives, owned in-repo rather than a black-box UI kit
Motion	motion (Framer) + react-spring: letter-by-letter blur-in headline, spinning circular text that 'goes bonkers' on hover, rotating 'do it? / find it? / snap it'	Memorable landing for a demo/hackathon
Navigation	Top bar: SaveMe wordmark · Send · Transactions · Tokens · Create · Chart · Connect	Five core actions, one click each
Feedback	react-hot-toast, status strings on /liquidity, 'Anomaly' badge and severity colours on transactions, service reachability in debug panel	User always knows what the chain and the ML service are doing

9. Next.js API routes

Route	Purpose	Validation and hardening
POST /api/anomaly	Proxy to FastAPI /detect	zod: 1-500 txs; numeric strings max 78 digits (uint256); hash max 66; 20 s AbortSignal timeout; 503 if unconfigured; 502 on upstream error; no detail leaked
POST /api/chat	Gemini 2.0 Flash NL transfer parser	zod 1-2000 chars; generateObject with schema {toAddress, amount, symbol}; system prompt treats user text as data only (prompt-injection guard); maxDuration 30
GET/POST /api/savecoin	Token metadata in Mongo (db memecoins, coll coins)	tokenAddress regex ^0x[a-fA-F0-9]{40}$ (blocks NoSQL operator injection); name ≤128, symbol ≤32, image ≤8 MB; upsert by address; ImgBB 20 s timeout
GET /api/savecoin/test	Health ping	Non-writing (used to be an open arbitrary insert)
GET /api/crypto	CoinMarketCap listings	Server-only COINMARKETCAP_API_KEY; revalidate 60; 10 s timeout; placeholder prices flagged fallback:true

`lib/mongodb.ts` exposes a lazy `getMongoClient()` that reads `MONGODB_URI` on first call, so a missing variable no longer kills `next build`.

10. ML anomaly-detection service



10.1 Pipeline

- **Input:** Pydantic `Transaction{hash, timeStamp, value, gas, gasPrice}` (strings, as Etherscan returns them), list bounded 1-1000.
- **DataCleaner:** drop duplicates (non-inplace + copy), fillna(0). `drop_invalid` is True for training and **False for scoring**, so every transaction sent gets a verdict.
- **DataTransformer:** converts UNIX timestamps only. Min-max normalisation was deliberately removed.
- **Features (5):** value, gas, gasPrice, value_per_gas = value/gas, total_gas_cost = gas·gasPrice. ±inf (gas = 0) → NaN → 0.
- **Scaling:** `log1p(clip(≥0))` then StandardScaler; `fit_transform` at training, `transform` at inference using the saved scaler.
- **Model:** `IsolationForest(contamination=0.01, random_state=42)` (100 trees, 256 samples per tree by default).
- **Rules:** 99.5th-percentile thresholds on value (severity high), gas and gasPrice (medium), value_per_gas (low).
- **Verdict:** `is_anomaly = bool(forest == -1) or bool(any rule breached)`. Forest-only hits are labelled `unusual_transaction_pattern` (medium).
- **Persistence:** joblib `isolation_forest / scaler / thresholds` in models/.

10.2 Why Isolation Forest

There is no labelled dataset of rug-pull transactions, so supervised learning is out. Isolation Forest isolates points with random axis-aligned splits; outliers need fewer splits, so their average path length h(x) is short. The score is s = 2^(−E[h(x)]/c(n)). It's linear-time, needs no distance metric, and works with a handful of features. Its weakness: it misses a point that is extreme on **one** axis but normal on the rest (a 900 ETH drain at 21,000 gas, 20 gwei). That's why the threshold rules run as a second, independent detector and the two are union'd.

10.3 Training data

`train_baseline.py` generates **4,000 seeded (42) synthetic transactions**: value log-normal (median 0.05 ETH, σ 1.4); gas 80% at 21,000 and 20% uniform 45k-250k; gasPrice log-normal (median 25 gwei, σ 0.6); inter-arrival exponential (mean 600 s). Log-normal because real transfer values and gas prices are right-skewed with long tails. It runs **during the Docker build**, so the pickle always matches the installed sklearn. You can replace it with real history via `POST /train` (body, or Etherscan with ETHERSCAN_API_KEY/ADDRESS).

10.4 Endpoints and service hardening

Endpoint	Behaviour
GET /	API info
POST /detect	400 if no model; returns results[], total_transactions, anomalous_transactions
POST /train	Needs X-Train-Key (hmac.compare_digest); 503 when TRAIN_API_KEY unset (fails closed); runs as BackgroundTask
GET /model/status	available + mtime + size, or 404
CORS	allow_credentials=False, methods GET/POST, headers Content-Type/X-Train-Key, origins from ALLOWED_ORIGINS
Errors	Global handlers log tracebacks server-side and return a generic 'Internal server error'

10.5 Measured quality

Metric	Before fixes	After fixes
Single-transaction request	500 (Input X contains NaN)	200
Recall on 3 obvious attacks (900 ETH drain, 1,200 ETH at 900 gwei, 3M-gas bomb)	0 / 3	3 / 3
False positives, 1,000 held-out normal txs (seed 1234)	0% (it flagged nothing)	3.0%
Same tx in different batches	Different verdicts	Identical verdicts

11. End-to-end user flows

A. Create a token

Connect wallet (RainbowKit) → /create, fill name/symbol/supply, optional logo (FileReader → base64) → wagmi `writeContract(TokenFactory.createToken)` → wallet signs → `useWaitForTransactionReceipt({query:{enabled}})` → parse `TokenCreated` log for the address → POST /api/savecoin → server validates, uploads to ImgBB, upserts into Mongo → appears on /tokens.

B. Natural-language transfer

'send 0.01 ETH to 0xabc...' → POST /api/chat → zod bound → Gemini `generateObject` returns {toAddress, amount, symbol} → client checks with viem `isAddress` (the LLM output is untrusted) → user confirms → `useSendTransaction` → explorer link built from the connected chain's `blockExplorers`.

C. Monitoring

/transactions → last 10 blocks fetched concurrently → user's txs with wei/gas/gasPrice → /api/anomaly → FastAPI pipeline → hash→verdict map → 'Anomaly' badge and per-type severity rows. Repeats every 30 s.

D. Emergency exit

Owner funds AnomalyGuardWallet (`forwardTokens` after approve, `forwardETH`) → on a red flag the owner calls `executeAnomalyExit()` → tokens swapped to ETH via the pool (or returned raw if the swap fails) → all ETH to owner. Today this trigger is manual and is not yet wired into the UI (see section 16).

E. Add liquidity

/liquidity → approve token → `addLiquidity.staticCall` to quote shares → minShares = 99% → send with ETH value.

12. Failures and how each was solved

Told in the order they were hit. For each: symptom → root cause → fix. Use these as your 'tell me about a hard bug' stories.

12.1 The app could not build

#	Symptom	Root cause	Fix
1	Bundle failed: missing getCallsStatus, sendCalls...	viem pinned at 2.23.3, but @wagmi/core 2.17 imports EIP-5792 actions added later. Peer range 'viem 2.x' so npm never warned	viem → ^2.55.13
2	TS errors inside viem/ox sources	tsconfig target ES5; libraries ship .ts using Map iteration and replaceAll (skipLibCheck only skips .d.ts)	target es2022, lib dom/dom.iterable/esnext
3	next build aborted without MONGODB_URI	mongodb.ts threw at module load; build imports every route to collect page data	Lazy getMongoClient()
4	/liquidity crashed	wagmi v1 API (useProvider), getSigner() during render	useWalletClient + ethers BrowserProvider, signer built in handler
5	log.args type error	ABIs not 'as const', so viem can't infer	as const on all five ABI files
6	Receipt hook ignored enabled	v1 option shape	query: { enabled }
7	ESLint failed build	3 rules-of-hooks violations in useLiquidityPoolFunctions.ts + 4 unescaped JSX entities	Entities escaped; the hook file had zero importers, so it was deleted
8	Prerender exploded locally	Node 25 defines a method-less globalThis.localStorage stub; WalletConnect's typeof check passes then crashes	engines.node 22.x for Vercel; NODE_OPTIONS=--no-experimental-webstorage locally

12.2 It built, but nothing would work in production

#	Problem	Fix
9	WalletConnect projectId was the literal 'YOUR_PROJECT_ID'	NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID (Reown Cloud)
10	Only transport was http://127.0.0.1:8545, so a deployed site called the visitor's machine	Default chain from env; Sepolia/Base Sepolia transports
11	Contract addresses hardcoded to Hardhat locals in 3 files	Centralised in lib/contract.ts, env-driven
12	Ignition modules hardcoded 0x5FbD…/0x9fE4…; deploy.sh targeted the in-process 'hardhat' network, discarded on exit	Single Guardiant.js reading the token from the TokenCreated event; deploy.sh takes a network and writes .env.local
13	initialSupply double-scaled → 10^42 tokens	Pass 1_000_000n; LP seed 10 ETH → 0.02 ETH
14	Explorer links hardcoded to mainnet etherscan.io	Derived from the connected chain's blockExplorers
15	Blank env vars: RainbowKit 'No projectId found', all 16 pages failed prerender	'' is not nullish, so ?? kept it; Number('') = 0. Switched 4 sites to ||

12.3 ML service: 200 OK that detected nothing

#	Problem	Root cause	Fix
16	Docker ran src/main.py	Not the API entrypoint; relative import beyond top-level; no PYTHONPATH	python:3.11-slim, PYTHONPATH=/app/src, uvicorn src.app:app
17	/detect 500 on fresh install	Committed joblib pickled with sklearn <1.3: 'node array has incompatible dtype'	Retrain during docker build
18	pip install failed on Python 3.13+	pandas 2.1.1 / numpy 1.24.3 have no wheels	Pin base image to 3.11
19	Single-tx request → 500 NaN; scores depended on batch	Min-max over the request batch (max == min → 0/0)	Removed; scaling belongs to the detector
20	Trained scaler ignored	prepare_features called fit_transform at inference	fit_scaler=False on load path
21	Forest couldn't isolate anything	Raw wei spans 10^15-10^21, heavy skew	log1p before scaling
22	Thresholds were noise	Trained on 3 rows of example_transactions.json	4,000-row baseline at build
23	900 ETH drain scored normal	Rules only consulted after the forest flagged a row	Union the detectors; thresholds 95th → 99.5th pct
24	Forest-only detections 500'd over HTTP only	np.bool_ or x returns np.bool_, which FastAPI can't serialise	bool() coercion + JSON-serialisability test
25	Approvals vanished from results	Zero-value rows dropped during cleaning	drop_invalid=False when scoring

12.4 Hugging Face Spaces deployment

Problem	Root cause	Fix
Image builds, Space would never boot	Spaces run as uid 1000; get_logger() opens logs/RotatingFileHandler at import → PermissionError on root-owned files	useradd -u 1000, COPY --chown=user, USER user before training
Build then failed at RUN python train_baseline.py	WORKDIR creates /app as root; the user owned the files but not the directory, so mkdir('logs') failed	mkdir -p /app && chown user /app (in working tree, not yet committed)
inf in features → 500 on /detect	value/gas with gas = 0	replace ±inf → NaN → 0 (in working tree, not yet committed)
model/ must be the Space repo root	It's a subdirectory of the monorepo	deploy-space.sh: git subtree split --prefix=model, force-push to Space only; idempotent
hf auth login failed (exit 1)	Interactive login not completed	Pending: run model/.venv/bin/hf auth login, then ./model/deploy-space.sh

12.5 Runtime performance

- **Runaway RPC loop (P1):** `useBlockNumber({watch:true})` in the effect deps re-ran the full 10-block scan every ~12 s block *and* every 30 s, serially, and re-fired the ML call each time. Fixed by reading the head inside the fetch and running the 10 calls concurrently.
- **Fabricated balances (P2):** '0.05 BTC' / '1.25 LINK' were hardcoded. Replaced by a real multicall; unreadable tokens show UNKNOWN.
- **Dead event listeners (P3):** watched TokenAdded/TokenRemoved, which the contract never emitted. Rewired to Deposited/Withdrawn.

13. Security audit: all 26 findings

ID	Sev.	Finding	Fix
S1	Critical	Linear swap pricing: swap = token reserve paid out 100% of ETH	Constant product + 0.3% fee
S2	Critical	LP positions ignored ETH: tokens + 1 wei withdrew others' ETH	sqrt shares, limiting side, proportional redemption
S3	High	Wallet.addToken credited balance with no transfer	Real custody; credit what arrived
S4	High	Exit reverted when balance large vs pool	try/catch swap, fall back to raw tokens
S5	High	.transfer() 2300 gas: contract-wallet owners locked out forever	call{value:} + require(ok)
S6	Medium	No slippage bounds	min* params everywhere
S7	Medium	No reentrancy guards; external call before state update	ReentrancyGuard; storage reserves
S8	Medium	Unbounded duplicate pushes in userTokens	O(1) membership map
S9	Medium	require(transferFrom) breaks on USDT-style tokens	SafeERC20
S10	Low	getSwapRate divided by zero on empty pool	Return 0
W1	High	NoSQL injection: {"tokenAddress":{"$ne":null}} overwrote any token	zod hex-address regex
W2	High	500s returned error.message + stack trace	Log server-side, generic message
W3	High	/api/savecoin/test unauthenticated arbitrary write	Non-writing ping
W4	Medium	CMC key in NEXT_PUBLIC_ var (would be bundled to browser)	COINMARKETCAP_API_KEY server-only
W5	Medium	Hardcoded prices served as 200, looked live	fallback: true flag
W6	Medium	No size limits on /api/chat, /api/anomaly	zod bounds
W7	Medium	Address check was prefix + length on an LLM-fed send path	viem isAddress()
M1	High	/train open on public Space: model poisoning	X-Train-Key, constant-time, fail closed
M2	High	Zero-value txs silently dropped	drop_invalid=False
M3	Medium	allow_origins * + credentials	Credentials off, narrowed
M4	Medium	np.bool_ serialisation 500	bool() + test
M5	Medium	Writes to DataFrame slices (no-op under copy-on-write)	.copy()
M6	Low	Unbounded /detect payload	max 1000
P1-P3	High-Low	RPC loop, fake balances, dead events (section 12.5)	See 12.5

14. Testing and verification

- **Contracts**: `contract/test/LiquidityPool.t.js` (6 tests, Hardhat + chai): swap cannot drain the pool's ETH; LP cannot withdraw ETH they never deposited; slippage bound honoured; cannot credit a balance without depositing; no duplicate token entries; guard still returns ETH when the pool swap fails. Run `npx hardhat test`.
- **ML**: `model/tests/test_detection_quality.py` (7 tests, plain python, no pytest): single tx doesn't crash; batch independence; drain at ordinary gas caught; JSON-serialisable with plain bools; zero-value txs scored; recall 3/3; FP rate < 5% (measured 3.0%). Run `cd model && python train_baseline.py && PYTHONPATH=src:. python tests/test_detection_quality.py`.
- **Probe-based debugging**: each ML bug was found by sending a deliberately shaped request (single row, drain, approval, mixed batch) over HTTP, not by reading code. M4 only reproduced over HTTP.
- **End-to-end**: local uvicorn + `next start` + `POST /api/anomaly` returned real scored results; Ignition deploy verified against a live local node, then on Sepolia.
- **Build**: `next build` went from hard failure to green; Vercel production deploys are Ready.

- Legacy `model/tests/test_*.py` need pytest, statsmodels, matplotlib, seaborn and dask, which are not in requirements.txt. The client has no test setup.

15. Deployment and hosting

Piece	Host	Status	How
client/	Vercel Hobby (project guardiant, root dir client)	Live, production Ready	`vercel --prod` from client/ (not git-connected yet)
contract/	Ethereum Sepolia	Deployed, all 6 + LP seeded	`./deploy.sh sepolia`
model/	HF Spaces Docker, port 7860	Pending	`hf auth login`, then `./model/deploy-space.sh`; set ANOMALY_API_URL on Vercel; set TRAIN_API_KEY secret
Metadata	MongoDB Atlas M0	Configured by env	Network access 0.0.0.0/0 (Vercel has no fixed egress IP)

Free-tier ceilings to know: Vercel 100 GB bandwidth, 100 GB-hrs functions/month; HF free CPU sleeps after 48 h idle (hence the 20 s proxy timeout); Atlas 512 MB; public Sepolia RPC is rate-limited; Gemini free tier has a per-minute cap.

Env vars (Vercel): NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID, NEXT_PUBLIC_DEFAULT_CHAIN_ID=11155111, NEXT_PUBLIC_{WALLET, TOKEN_FACTORY, LIQUIDITY_POOL, TRANSFER, ANOMALY_GUARD, TOKEN_FACTORY_LP}_ADDRESS, ANOMALY_API_URL, MONGODB_URI, GOOGLE_GENERATIVE_AI_API_KEY, IMGBB_API_KEY, COINMARKETCAP_API_KEY.

16. Known limitations (be honest about these)

- **The exit isn't automatic.** `executeAnomalyExit` is onlyOwner and nothing on-chain listens to the ML service. Automating it needs a keeper (Chainlink Automation / Gelato) or a bot holding a limited 'guardian' role.
- **AnomalyGuardWallet isn't wired into the UI.** No page calls it yet.
- **Stale ABI on /liquidity.** `client/src/abis/LiquidityPool.json` still declares `addLiquidity(tokenAmount)` with one argument, while the page and the contract use `(tokenAmount, minShares)`. ethers will reject the 2-argument call. Fix: regenerate that JSON from the compiled artifact.
- **Synthetic baseline.** The model learns generic 'normal', not this wallet's normal; /train on the Space is ephemeral and resets on restart.
- **Per-transaction only.** Sequence patterns (many small drains, approval bursts) need windowed features: rate, count, cumulative outflow.
- **10-block lookback** (~2 minutes on Sepolia). A real fix uses Etherscan `txlist` or an indexer.
- **/graph is a simulation,** not connected to the ML service.
- **Two fixes uncommitted:** the Dockerfile /app chown and the inf guard in isolation_forest.py.
- DEPLOYMENT.md Part 3 items 1, 5 and 6 (mock balances, linear AMM, duplicate pushes) are outdated; the audit already fixed them.

17. Likely interview questions with answers

Q. Why Isolation Forest and not a neural network or a supervised classifier?

No labelled rug-pull data exists, so it has to be unsupervised. IF is linear-time, needs no distance metric, is robust with 5 features, and is explainable enough to pair with rule-based reasons. A deep autoencoder would need far more data for no clear win.

Q. Why combine the forest with threshold rules?
They fail differently. The forest finds unusual combinations but under-weights a single extreme feature; a 900 ETH drain at normal gas looked normal to it. Rules catch single-axis extremes and give a human-readable reason. A union maximises recall for a safety product, and 99.5th-percentile thresholds keep false positives near 3%.

Q. Why log1p before StandardScaler?
Wei values span six orders of magnitude and are right-skewed. Standardising raw values leaves every row except the largest within a fraction of an SD of the mean, so nothing can be isolated. Log brings them onto a scale where 'far' means something.

Q. Why must the scaler be fitted only at training time?
Otherwise a score depends on what else is in the request. The same transaction got different verdicts in different batches, and a one-row batch divides by zero. A model has to be a fixed function of its input.

Q. How do you evaluate without labels?
Synthetic held-out normal traffic from a different seed for false positives (3.0%), plus hand-built attack archetypes for recall (3/3). I'd be clear these are sanity checks, not production metrics; real evaluation needs labelled incidents.

Q. Explain x·y=k and why the old pool was exploitable.
Price comes from the reserve ratio and moves against you as you trade, so output approaches but never reaches the reserve. The linear formula priced every unit at the spot price, so selling an amount equal to the token reserve took 100% of the ETH.

Q. Why MINIMUM_LIQUIDITY?
Burn 1000 shares on the first deposit so total supply never returns to zero. That prevents re-initialising the pool at a manipulated price and the share-inflation attack on tiny pools.

Q. Why call{value:} instead of transfer()?
transfer forwards 2300 gas, which isn't enough for smart-contract wallets like Safe. For an emergency exit that would mean funds locked forever. call forwards all gas, so it's paired with ReentrancyGuard and a success check.

Q. Why minEthOut = 0 in the exit? Isn't that a sandwich risk?
Yes, deliberately. In a panic, a reverting exit is worse than a bad price. Front-running costs value but cannot block the withdrawal. A production version could take an owner-supplied floor with a fallback.

Q. Why storage reserves instead of balanceOf?
Anyone can send tokens or ETH directly to a contract. If price read balanceOf, a donation would move the price, which is a manipulation vector. Storage reserves only change through the pool's own functions.

Q. How did you find the NoSQL injection?
tokenAddress went from JSON straight into findOne/updateOne. JSON can carry objects, so {"$ne": null} is a valid Mongo operator that matches the first document. The fix is validating to a primitive hex string at the trust boundary with zod.

Q. How do you stop the LLM sending funds to the wrong place?
Three layers. A system prompt treats user text as data. generateObject constrains output to a zod schema. Then the client checks the address with viem isAddress, and the user confirms in their wallet. The LLM output is never trusted.

Q. Why proxy the ML API through Next.js?
It hides the service URL, avoids CORS and mixed content, lets us validate and bound input before spending free-tier CPU, and adds a timeout for a sleeping Space.

Q. What was the hardest bug?
The np.bool_ one. Direct function calls passed every check, but over HTTP any forest-only detection 500'd, because `np.bool_ or x` returns the numpy object, which FastAPI can't serialise. Only an HTTP-level probe caught it, so a JSON-serialisability test now pins it. Runner-up: the ML returning 200 while detecting nothing at all, caused by four separate feature-path bugs.

Q. Why '||' instead of '??' for env vars?

A key that's present but blank arrives as "", which is not nullish, so ?? keeps it. RainbowKit then threw 'No projectId found' and all 16 pages failed prerender; Number("") gave chain id 0.

Q. Why train the model inside the Docker build?

joblib pickles aren't forward-compatible across sklearn minor versions; the committed model crashed under 1.3.1. Training at build time guarantees the artifact matches the runtime library, and the Space works from a cold start.

Q. Why Hardhat Ignition over a deploy script?

It models deployment as a dependency graph, resumes from its journal if a tx fails mid-way (handy with faucet ETH), and can read event arguments, such as the new token address from TokenCreated, as inputs to later steps.

Q. Why wagmi/viem over plain ethers?

Typed React hooks with caching through TanStack Query, ABI-level type inference from 'as const', and a smaller bundle. ethers is kept only where its BrowserProvider + staticCall made the /liquidity flow simpler.

Q. How would you make it production-grade?

A keeper or guardian role to trigger exits automatically; windowed sequence features and per-wallet models trained on real Etherscan history stored in object storage; an indexer instead of block scanning; an external audit and fuzzing (Foundry) of the pool; auth and rate limiting on the API routes; CI running both test suites.

Q. What would you do differently?

Write the regression tests before the features, keep ABIs generated from build artifacts (the stale /liquidity ABI shows why), and validate ML with probes on day one: a service that returns 200 isn't a service that works.

18. Glossary and numbers to remember

Term / number	Meaning
Rug pull	Developers or insiders drain liquidity or holdings, leaving holders with worthless tokens
AMM, x·y=k	Automated market maker; product of reserves stays constant (pre-fee)
LP shares	Claim on a proportional slice of both reserves
Slippage bound	Minimum acceptable output; the tx reverts if the price moved
Reentrancy	External call re-enters the contract before state is updated
SafeERC20 / forceApprove	Wrappers that handle tokens returning no bool, and allowance-reset quirks
EIP-5792	Wallet batch-call API (sendCalls) that caused the viem version break
Contamination 0.01	IF's expected outlier fraction, which sets its decision threshold
4,000 / seed 42	Baseline training rows
99.5th pct	Rule thresholds
3/3, 3.0%	Recall / FP rate
0.3% (997/1000)	Swap fee
1000	MINIMUM_LIQUIDITY
10,000 GDT + 0.02 ETH	Pool seed
1,000,000 GDT	Demo token supply
≈ 7.49M gas	Full protocol deploy
11155111	Sepolia chain id
7860	HF Space port
1-500 / 1-1000	Tx bounds: proxy / FastAPI
20 s	Proxy timeout
10 blocks / 30 s	History lookback / poll
26 / 13	Audit findings fixed (10 contract, 7 web, 6 ML, 3 perf) / regression tests (6 contract + 7 ML)